BhavikBarai / billetto_rails_test.md Secret

Forked from kaspergrubbe/billetto_rails_test.md
Last active now

Code Revisions 5

billetto_rails_test.md

Billetto Ruby on Rails Engineer Test

This test assesses a candidate's proficiency in the Ruby on Rails framework, their ability to work with external APIs, set up an event-driven architecture using Rails Event Store, implement user authentication with Clerk.com, and write effective tests.

Objective

The candidate will build a Rails application that:

1. Fetches and ingests events data from the Billetto API.
2. Organizes and displays the events.
3. Implements a simple voting feature using Rails Event Store, https://railseventstore.org/.
4. Adds user authentication using Clerk.com, https://clerk.com/.

Test Structure

1. Setting up the Application and Integrating with Billetto API

- **Objective**: Set up a Rails application, connect to an external API, and handle data ingestion and presentation.
- **Task Requirements**:
  - **Environment Setup**: Set up a basic Rails application using any data source (Billetto uses MySQL, but we also really like PostgreSQL; the choice is yours!).
  - **API Integration**: Create a service to fetch event data from the Billetto API with error handling for failed requests and data validation.

    - **Data Storage**: Ingest the fetched events into a model (e.g., `Event`) in the Rails application, ensuring data consistency and necessary validations.
    - **Data Display**: Build a simple page to list events in an organized manner, including title, date, image, and a brief description.

The documentation for the Billetto API endpoint is available here: https://api.billetto.com/reference/list-public-events. You will need to create a user and set up an API-key pair to access the events.

2. Adding Clerk.com Authentication

- **Objective**: Set up user authentication to restrict voting functionality to logged-in users only.
- **Task Requirements**:
    - **Clerk.com Setup**: Integrate Clerk.com for user authentication. Set up sign-up, login, and logout functionality using Clerk's SDK.
    - **User Restrictions**: Restrict voting functionality so that only authenticated users can vote on events.
    - **User Association**: Associate votes with specific users. Store user details (e.g., user ID) along with vote events in the Rails Event Store for future traceability.

3. Setting up Rails Event Store

- **Objective**: Set up and use Rails Event Store for a voting feature, showcasing an understanding of event-driven architecture.
- **Task Requirements**:
    - **Rails Event Store Setup**: Install and configure Rails Event Store, including brief setup documentation.
    - **Voting Feature**: Implement a voting mechanism on events:
        - Each event should support voting (like or dislike).
        - Use Rails Event Store to record votes as events (e.g., `EventUpvoted`, `EventDownvoted`) along with the associated user's ID.
        - Display the total count of votes for each event on the event listing page.

4. Testing

- **Objective**: Verify the candidate's ability to write effective tests using RSpec or Minitest.

- **Task Requirements**:
  - **Model Tests**: Ensure data integrity in the `Event` model with validations (e.g., title presence, date validation).
  - **Authentication Tests**: Test that voting functionality is restricted to authenticated users.
  - **Event Store Tests**: Test the voting feature, verifying voting events are created and counted correctly.
  - **Browser Test**: Test the authentication flow (sign-up, login, logout) and voting functionality using a browser.

Evaluation Criteria

1. **Technical Proficiency**:
   - Ability to set up a Rails application with a data source.
   - Proficiency in working with external APIs and handling data ingestion.
   - Proper setup and use of Rails Event Store for event management.
   - Integration of authentication using Clerk.com.
2. **Code Quality**:
   - Clean, readable, and maintainable code.
   - Clear separation of concerns and adherence to best practices.
   - Effective error handling and data validation.
3. **Testing**:
   - Coverage and effectiveness of tests.
   - Proper usage of RSpec/Minitest to ensure reliability of features.
4. **Documentation**:
   - Short setup instructions for their environment.
   - Brief explanation of design choices and any assumptions made during development.

Submission Guidelines

The candidate should submit:

- A link to the GitHub repository containing the code.
- Any setup instructions in the README.md file.

At the follow-up interview we expect:

- A brief summary of their approach, design choices, and assumptions.

BhavikBarai commented now Author